Chapter 9: Lists and Indexing

Gregory S. DeLozier, PhD

Every value this interpreter has handled so far has been a single, immutable thing: a number, a string, a boolean. Even a variable holding one of these can only ever be replaced, never changed from the inside. A **list** is different on both counts: it holds many values at once, in order, and it can be modified in place without being replaced. (1) That second property is new enough to the language that it changes what assignment itself has to mean.

```
scores = [70, 80, 90];
scores[0] = 75;
print(scores);
```

This prints `[75, 80, 90]`. The list `scores` was not replaced; its first element was.

Breaking Changes

Earlier use	Change
A variable, or any identifier, named `length`, `slice`, `head`, or `tail`	Rename it; each is now a keyword.
A program using `[` or `]` for any other purpose	Not previously legal syntax; now reserved for list literals and indexing.

Nothing about numbers, strings, booleans, conditionals, or loops changes. Lists are a new kind of value, built on everything already established, not a replacement for any of it.

List Literals

```
empty = [];
mixed = [1, "two", true];
```

A list literal is a comma-separated sequence of expressions between square brackets, including zero of them. The elements do not need to share a type; `mixed` holds a number, a string, and a boolean at once.

```python
if ast["tag"] == "list":
    items = []
    for element in ast["items"]:
        value, status = evaluate(element, environment)
        if status is not None:
            return value, status
        items.append(value)
    return items, None
```

Each element is evaluated in source order, and its status is checked before moving to the next one, the same discipline every other multi-part construct in this interpreter already follows. `[1, exit(7), 3]` never evaluates the `3`.

Reading by Index

```
numbers = [10, 20, 30];
print(numbers[1]);
```

This prints `20`. An index is a position, counted from `0`; it is not the same thing as the element found there, and mixing the two up is the most common mistake a reader of this chapter will make at first. `numbers[1]` is the element at position `1`, which happens to be the number `20`.

Indexing is **postfix**: `expr[expr]` applies after any expression that produces a list or a string, not only an identifier. The parser accepts an index suffix after any factor, so `[10, 20, 30][1]` is legal, if unusual, and so is `"hello"[1]`.

```
def parse_complex_suffixes(node, tokens):
    # { index_suffix }, applied to a node already parsed. Shared by reading
    # (postfix indexing in an expression) and writing (an assignment
    # target): both are the same chain of "complex" nodes, evaluated
    # differently depending on which side of "=" they end up on. The tag is
    # "complex" rather than "index" on purpose: a later chapter's function
    # calls are the same postfix shape with a different suffix, and will
    # extend this same node instead of introducing a parallel one.
    while tokens[0]["tag"] == "[":
        index, tokens = parse_expression(tokens[1:])
        tokens = require(tokens, "]", "Expected ']' after index")
        node = {"tag": "complex", "base": node, "index": index}
    return node, tokens
```

The evaluator checks status from both the base and the index before touching either one, the same two-child pattern arithmetic has used since the pair of them first appeared:

```
if ast["tag"] == "complex":
    # Reading: base and index are both evaluated and checked for status
    # before the access happens. Assignment to the same node shape is
    # handled separately, under "assign", where this same read logic
    # is what resolves every level except the last.
    base, status = evaluate(ast["base"], environment)
    if status is not None:
        return base, status
    index, status = evaluate(ast["index"], environment)
    if status is not None:
        return index, status
    # A string can be read by index, not assigned. Reading yields a
    # one-character string; Vertex has no separate character type.
    if type(base) not in (list, str):
        raise TypeError("Indexing requires a list or string")
    return base[require_index(index, base)], None
```

An index must be an integer, or a float with no fractional part: `3.0` is accepted the same as `3`, but `3.5` is not. Negative indices are rejected outright; this chapter never counts backward from the end of a list. Python treats `bool` as a subtype of `int`, so `require_index` excludes it by checking the exact type, not just whether the value happens to behave like one:

```
def require_index(index, base):
    # An index must be an integer, or a float with no fractional part:
    # 3.0 is accepted, 3.5 is not. Python treats bool as a subtype of int,
    # so it is excluded explicitly rather than by accident. No negative
    # indices, and no silent growth on assignment: both directions of
    # out-of-range access raise the same error. The same rules apply to a
    # list and to a string.
    if type(index) is bool or type(index) not in (int, float):
        raise TypeError("Index must be an integer")
    if type(index) is float and not index.is_integer():
        raise TypeError("Index must be an integer")
    index = int(index)
    if not (0 <= index < len(base)):
        raise IndexError("Index out of range")
    return index
```

`numbers[true]` and `numbers[1.5]` are both type errors. `numbers[3]`, on a three-element list, is out of range: a clear failure, not three implicitly becoming the list's new length.

Strings can be indexed the same way. `"hello"[1]` is `"e"`, a one-character string, because Vertex has no separate character type. Every rule above applies unchanged: whole-number indices only, no negatives, and an out-of-range index, including any index into an empty string, is an error. This is deliberate. A language should not be a pile of special cases. Once you know how a list is indexed, you can predict how a string is, and the same holds for `length`, `slice`, `head`, and `tail`. The two differ in one place, assignment, and there the reason is real: a string cannot be changed in place.

Chained Indexing

```
matrix = [[1, 2], [3, 4], [5, 6]];
print(matrix[1][0]);
```

This prints `3`. `matrix[1]` is the list `[3, 4]`; indexing that result with `[0]` reads its first element. There is no special two-argument indexing form for this. `matrix[1][0]` is two `"complex"` nodes, the outer one using the inner one as its base:

```
{
    "tag": "complex",
    "base": {
        "tag": "complex",
        "base": {"tag": "identifier", "value": "matrix"},
        "index": {"tag": "number", "value": 1},
    },
    "index": {"tag": "number", "value": 0},
}
```

Evaluating the outer node evaluates its base first, which is itself a `"complex"` node and gets evaluated the same way any expression would be. Nothing about chained indexing needed new evaluator code.

Traversal with While and Length

```
values = [3, 1, 4, 1, 5, 9];
total = 0;
i = 0;
while (i < length(values)) {
    total = total + values[i];
    i = i + 1;
}
```

`total` ends at `23`. `length` is a dedicated expression, parenthesized like `number()` or `type()`, not a general function call; Vertex does not have those yet.

```
if ast["tag"] == "length":
    value, status = evaluate(ast["expression"], environment)
    if status is not None:
        return value, status
    # Lists count elements; strings count characters.
    if type(value) not in (list, str):
        raise TypeError("length argument must be a list or string")
    return len(value), None
```

`length` accepts a list or a string. A list's length is its number of elements and a string's length is its number of characters, so `length("abc")` is `3` and `length("")` is `0`. Any other type is a type error.

Slice, Head, and Tail

```
slice_expression ::= "slice" "(" expression "," expression "," expression ")"
head_expression ::= "head" "(" expression ")"
tail_expression ::= "tail" "(" expression ")"
```

```
letters = [10, 20, 30, 40];
print(slice(letters, 1, 3));
print(head(letters));
print(tail(letters));
```

This prints `[20, 30]`, then `10`, then `[20, 30, 40]`. Like `length`, these are dedicated expressions that look like calls but are not function calls. Each accepts a list or a string, and on a string each works on characters and returns a string: `slice("hello", 1, 3)` is `"el"`, `head("hello")` is `"h"`, and `tail("hello")` is `"ello"`.

`slice(sequence, start, end)` is half-open. It includes the element at `start` and stops before the element at `end`, so the result has `end - start` elements (characters, for a string). The result is always new: a new outer list for a list, a new string for a string. Positions count from `0`, as they do for indexing. JavaScript and Python use the same convention. (2) `slice(t, 0, length(t))` is the whole sequence, and `slice(t, 2, 2)` is empty. The two bounds follow the indexing rules: an integer, or a float with no fractional part, and never a boolean. They differ in

one way. An index must name an element, but a bound may equal the length, because the end position is where the slice stops, not an element. That rule applies to `start` as much as to `end`: `slice([10, 20, 30, 40, 50, 60], 6, 6)` is `[]`, not an error. `start` looks like it names a seventh element that does not exist, but it does not name anything; it only says where the slice begins, the same way `end` only says where it stops. Both bounds sitting at the length is the same situation `slice(t, 0, 0)` already is at the front: a valid position to start and stop at once, just with nothing between. A bound of the wrong type, such as a boolean or a number with a fractional part, raises `TypeError`. A negative bound, a bound greater than the length, or a start greater than the end raises `IndexError`. Some languages clamp out-of-range bounds quietly. Vertex fails the same way an out-of-range index does.

`head` returns the first element, or the first character as a one-character string. `tail` returns everything after it. For a list, the result is a new outer list. For a string, it is a new string, which cannot be told apart from the original except by value, because strings never change in place.

`head` rejects an empty list or string with an `IndexError`: there is no first element to return, and Vertex has no null value to stand in for one. `tail` does not. `tail` is `value[1:]` underneath, and Python's own slicing already agrees that the tail of nothing is nothing: `[][1:]` is `[]`, with no error. `tail([])` is `[]` and `tail("")` is `""`. A loop that calls `head` still needs to check `length` first:

```
remaining = [3, 1, 2];
backwards = [];
while (length(remaining) > 0) {
    backwards = [head(remaining)] + backwards;
    remaining = tail(remaining);
}
```

`backwards` ends as `[2, 1, 3]`. Each pass moves the first element of `remaining` to the front of `backwards`, and the `length` check stops the loop before `head` ever sees an empty list. `tail` runs on the last, single-element `remaining` without needing that check, though the loop still needs it for `head`.

```
if ast["tag"] in ("head", "tail"):
    value, status = evaluate(ast["expression"], environment)
    if status is not None:
        return value, status
    if type(value) not in (list, str):
        raise TypeError(f"{ast['tag']} argument must be a list or string")
    if ast["tag"] == "head":
        if len(value) == 0:
            kind = "list" if type(value) is list else "string"
            raise IndexError(f"head of an empty {kind}")
        return value[0], None
    return value[1:], None
```

```
def require_slice_bound(bound, sequence):
    if type(bound) is bool or type(bound) not in (int, float):
        raise TypeError("Slice bounds must be integers")
    if type(bound) is float and not bound.is_integer():
        raise TypeError("Slice bounds must be integers")
    bound = int(bound)
    if not (0 <= bound <= len(sequence)):
        raise IndexError("Slice bound out of range")
    return bound
```

`slice` evaluates its sequence, then its start, then its end, checking status after each, before it checks any type or range. An `exit` in any argument propagates before the others are touched. Like concatenation, `slice` and `tail` build a new outer list, so changing the result never changes the original. The elements are shared, though. After `b = slice(a, 0, 1);` on `a = [[1], [2]]`, the statement `b[0][0] = 99;` also changes `a[0][0]`, because `b[0]` and `a[0]` are the same inner list.

Indexed Assignment

```
assignment_statement ::= <identifier> { index_suffix } "=" expression
```

An assignment target is an identifier optionally followed by one or more `[index]` suffixes, parsed with the exact same `parse_complex_suffixes` helper that builds a read expression. The difference between reading and writing is not in the grammar; it is in what the evaluator does with the resulting node depending on which side of `=` it ends up on.

```
if ast["tag"] == "assign":
    # The target's own subexpressions, if it has any, are resolved before
    # the right-hand value. For an identifier target that resolves
    # nothing (just a name), so this is the same order as before. For an
    # indexed target, the base and the index are both evaluated, and
    # checked for status, ahead of the value.
    target = ast["target"]
    if target["tag"] == "identifier":
        name = target["value"]
        destination = environment
        if name in ("__input", "__output"):
            destination = global_environment(environment)
        key = name
    elif target["tag"] == "complex":
        base, status = evaluate(target["base"], environment)
        if status is not None:
            return base, status
        index, status = evaluate(target["index"], environment)
        if status is not None:
            return index, status
        if type(base) is not list:
            raise TypeError("Indexed assignment requires a list")
        key = require_index(index, base)
        destination = base
    else:
        raise ValueError("Assignment requires an identifier or indexed destination")
    value, status = evaluate(ast["expression"], environment)
    if status is not None:
        return value, status
    destination[key] = value
```

The order is deliberate, and worth stating plainly: **the target is resolved before the right-hand value is evaluated**, for every kind of target. For a plain identifier that resolution does nothing observable, so the order was never visible before this chapter. For an indexed target it is: given `a = [1, 2, 3]`, `a[exit(6)] = exit(999)` returns `(6, "exit")`, not `(999, "exit")`, because the index is checked before the value is ever evaluated. If the base of an indexed target is itself an indexed read, as in a chained target, that read's own status is checked the same way, before the outer index and before the value. Nothing is stored, anywhere along the chain, once any of this returns a status: `a[0] = exit(9);` leaves `a` byte-for-byte what it was.

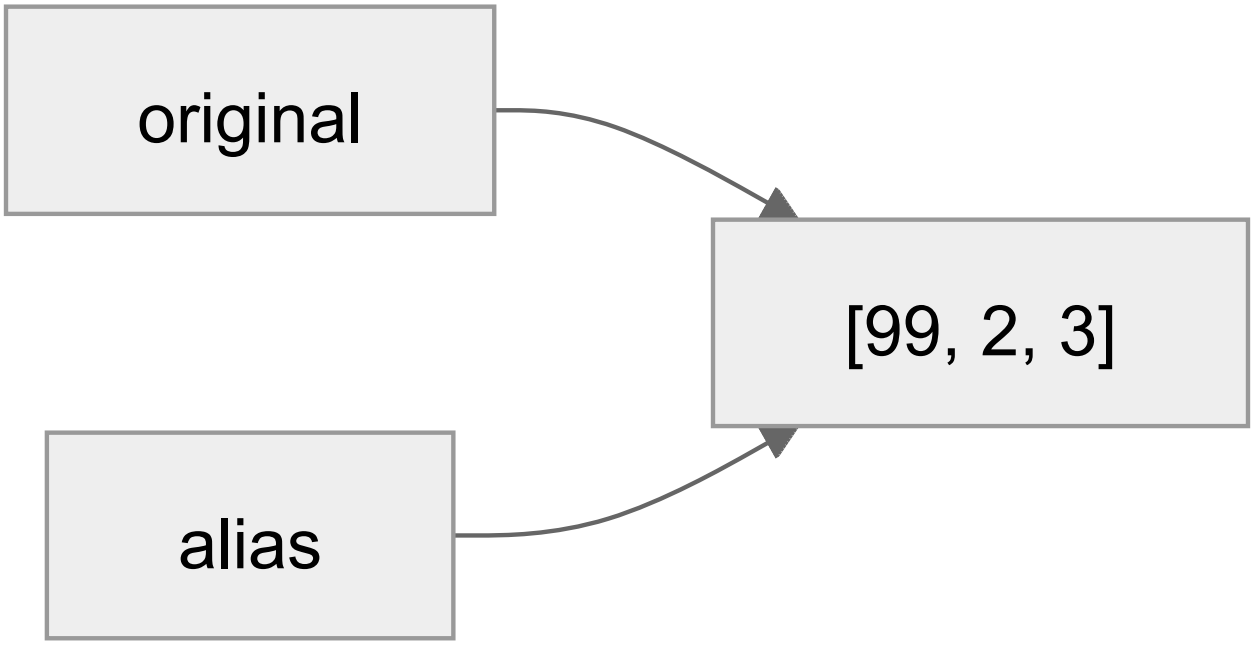
Out-of-range assignment is rejected the same way an out-of-range read is: `a[5] = 1;` on a three-element list raises an error. It does not extend the list to make room.

Only a list can be an assignment target. `word[0] = "j";` on a string is a type error, because a string can only be replaced, never changed from the inside. To get the new string, build it: `word = "j" + tail(word);`.

Mutation and Rebinding

```
original = [1, 2, 3];
alias = original;
alias[0] = 99;
print(original);
```

This prints `[99, 2, 3]`. `alias = original` did not copy the list. Assignment binds a name to a value, and here it bound a second name to the same list that `original` names. Numbers and strings never exposed this, because they cannot be changed in place, only replaced: two names sharing a number behave exactly like two names holding separate copies of it. A list can be changed in place, so two names can now share one list, and a mutation through either name is visible through both. No new interpreter code made this possible. The environment has always mapped a name to a value; a list is simply the first value where sharing becomes observable.

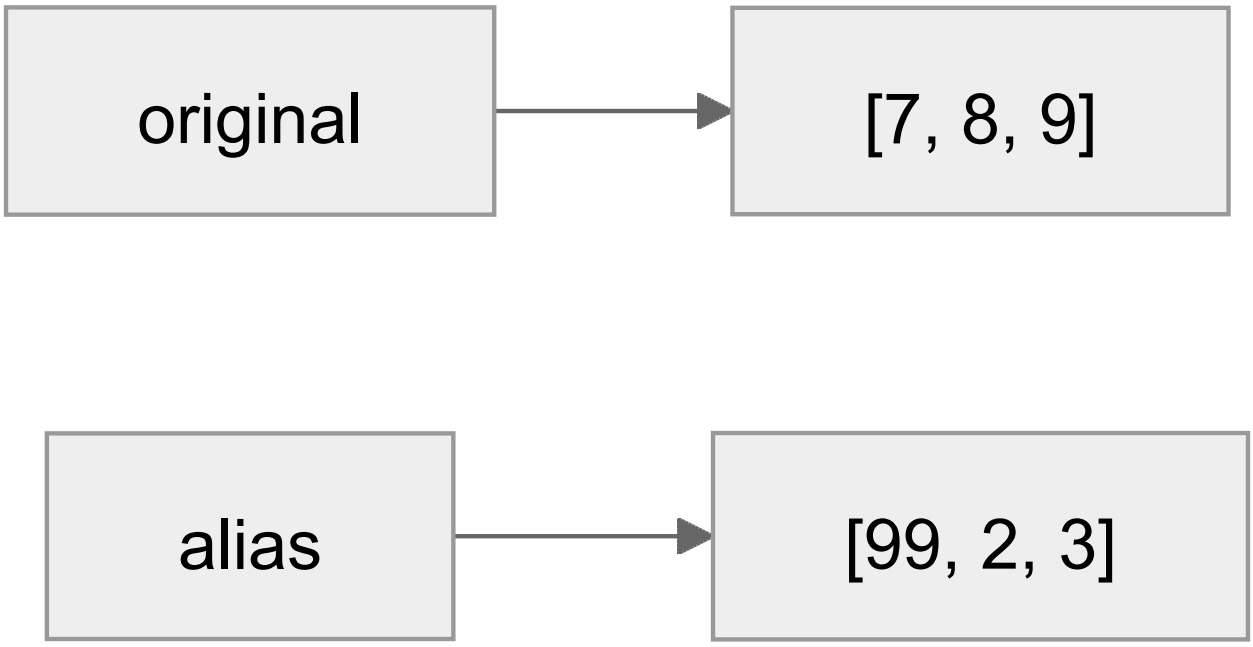


Both names point at the same list. There is only one list here, not two copies that happen to match.

Rebinding is different from mutating, and the two must not be confused. Continuing from the example above, where `original` and `alias` name the same list:

```
original = [7, 8, 9];
print(alias);
```

This still prints `[99, 2, 3]`. Assigning a new list to `original` gives that name a new list to point to. It says nothing about what `alias` points to, which has not changed.



`original` now points somewhere new. `alias` was never touched by that assignment and still points at the list it always did.

Indexed assignment's target is resolved by reading it, and reading an identifier has always searched outward through enclosing environments, not just the current one, since Chapter 6. Nothing in this chapter had to decide that; `x[10] = 4` simply reads `x` to find the list it names, wherever that reading finds it, then mutates that list in place, exactly as `alias[0] = 99` does above. Nothing in the language before Chapter 11 actually creates an enclosing environment for a program to try this with, but the consequence already follows from code this chapter did not change: once one exists, `x[10] = 4` will reach outward and mutate a list an enclosing scope owns, without creating a local `x` and without rebinding anything, no special declaration required. Plain rebinding is different, and stays different: `x = [9, 9]` always writes to the current environment, creating a new local `x` that shadows the outer one rather than replacing it, the same local-write rule Chapter 2 always used. What it would take for a rebinding to reach outward instead is a question for a later chapter, not this one.

Equality

```
print([1, 2] == [1, 2]);
print([true] == [1]);
```

This prints `true`, then `false`. Two lists are equal when they have the same length and their elements are equal pairwise, checked with this same rule recursively, not with Python's own ==:

```
def vertex_equal(left, right, comparing=None):
    # Lists compare structurally, recursively applying this same rule to
    # every element, rather than Python's own ==, which would let a
    # nested true and 1 compare equal and break the Chapter 4 rule that
    # they never do.
    #
    # Two extra mechanisms guard against indexed assignment making a list
    # contain itself (a[0] = a). The identity check is the common case
    # and the cheap one: two names for the same list are equal without
    # looking at a single element. It is not enough on its own, though:
    # two separately built cyclic lists (a=[0]; a[0]=a; b=[0]; b[0]=b)
    # are never identical, so comparing them would still recurse forever
    # without comparing is tracking which pairs are already being
    # compared on the path from the outermost call to here. Reaching the
    # same pair again means nothing left to compare could ever disagree,
    # so it counts as equal and the recursion stops. comparing is a
    # plain set, mutated with add before recursing and discard after, so
    # each list pair costs one O(1) membership test, not a copy of the
    # whole set at every level.
    if left is right:
        return True
    if type(left) is list and type(right) is list:
        if comparing is None:
            comparing = set()
        pair = (id(left), id(right))
        if pair in comparing:
            return True
        if len(left) != len(right):
            return False
        comparing.add(pair)
        try:
            return all(vertex_equal(l, r, comparing) for l, r in zip(left, right))
        finally:
            comparing.discard(pair)
    numeric = type(left) in (int, float) and type(right) in (int, float)
    return (numeric or type(left) is type(right)) and left == right
```

Python's own `[True] == [1]` is `True`, because Python considers `True` and `1` numerically equal everywhere, including inside a list. Vertex decided back in Chapter 4 that a boolean and a number are never equal, `true != 1`, and that rule has to keep holding no matter how deeply a value is nested inside a list for it to mean anything. `vertex_equal` is what makes that true recursively instead of only at the top level.

Ordering is different, and the difference is not a special case. <, <=, >, and >= are defined only where a value has a known order. Numbers have one. Strings have one: character order, compared from left to right. Lists have none. Is `[1, 2]` less than `[1, 3]`? Is `[1]` less than `["a"]`? Vertex does not invent an answer, so `[1] < [2]` is a type error. The same rule rejects mixed types. `"a" < 4` is an error, and so is `4 < "a"`, because no order relates a string to a number. Equality needs no order, only a way to tell whether two values are the same, so == works on any two values and `"a" == 4` is simply `false`. The evaluator's operand check keeps Python's own list ordering from becoming Vertex behavior by accident.

Concatenation

```
first = [1, 2];
second = [3, 4];
combined = first + second;
combined[0] = 99;
print(first);
```

This prints `[1, 2]`: `combined` is a new list, not a third name for `first`'s elements followed by `second`'s. `+` already overloaded for numbers and strings; extending it to two lists is the same kind of overload, and Python's own list `+` already builds a fresh list rather than sharing either operand, which is exactly the copy behavior concatenation needs here. No other arithmetic operator accepts a list on either side: `[1, 2] - [3]`, `[1] * 2`, and `[1, 2] + 3` are all type errors.

`+` joins two lists and never appends a single element. If it did, `[1] + [2]` would be ambiguous: it could mean joining two lists, giving `[1, 2]`, or appending the list `[2]` as one element, giving `[1, [2]]`. Requiring a list on both sides leaves one reading. To append an element, wrap it in a list first: `[1] + [x]`.

Printing and Conversions

```
print([1, "hi", true, [2, 3]]);
```

This prints `[1, "hi", true, [2, 3]]`. Printing a bare string still prints it unquoted, as it always has; a string sitting inside a list needs quotes, or `["hi"]` and `[hi]` would look the same with no way to tell an element that happens to be text apart from one that was never a string in the first place.

```
def vertex_repr(value, ancestors=None):
    # The display form used inside a list, where a bare string needs
    # quotes to stay distinguishable from everything around it. print()
    # and string() use this for a list; a string or number printed on its
    # own still uses its plain, unquoted form.
    #
    # Identity alone cannot stop a cyclic list the way it does in
    # vertex_equal: printing one still has to describe an element that
    # is the list itself. ancestors tracks which lists are currently
    # being printed, on the path from the outermost call down to here,
    # the same add-before/discard-after pattern vertex_equal uses:
    # reaching one again prints "..." instead of recursing into it
    # again, and discarding on the way back out means two separate,
    # non-cyclic appearances of the same list still both print in full.
    if type(value) is bool:
        return "true" if value else "false"
    if type(value) is list:
        if ancestors is None:
            ancestors = set()
        if id(value) in ancestors:
            return "[...]"
        ancestors.add(id(value))
        try:
            return "[" + ", ".join(vertex_repr(item, ancestors) for item in value) + "]"
        finally:
            ancestors.discard(id(value))
    if type(value) is str:
        escaped = (value.replace("\\", "\\\\").replace('"', '\\"')
                   .replace("\n", "\\n").replace("\t", "\\t").replace("\r", "\\r"))
        return '"' + escaped + '"'
    return str(value)
```

`string()` uses the same representation for a list, so `string([1, 2])` is `"[1, 2]"`. `type([1, 2])` is `"list"`. `number()` and `boolean()` both reject a list outright: there is no sensible single number or truth value a list should become, so neither conversion pretends there is one.

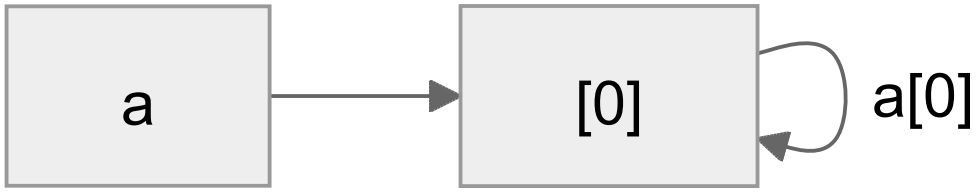
Self-Reference

Indexed assignment accepts any value as the right-hand side, including the list being assigned into:

```
a = [0];
a[0] = a;
```

`a[0]` is now the same list `a` names, not a copy of it. Nothing new makes this possible; it is the same sharing `alias = original` relies on in the previous section, reached through an index instead of a second name. Nothing in the parser or evaluator treats “a list containing itself” as a case to recognize, because nothing ever asks whether a value and the place it is being stored are related.

`vertex_equal` and `vertex_repr` both walk a list's elements recursively, and a self-referential list gives that walk no bottom: the first element of `a` is `a`, so walking into it only produces `a` again. Without the mechanisms described below, `a == a` and `print(a)` would recurse until Python's own call stack gave out, crashing with a raw `RecursionError` instead of producing any Vertex-level result.



Comparing `a` to itself is the cheap case: checking `left is right` first means two names for the same list are equal immediately, before any element is examined, and this is exactly how Python's own list == already behaves, which is why plain Python has no trouble with `a=[0]; a[0]=a; a==a`. Identity alone is not enough in general, though. Two separately built cycles, `a=[0]; a[0]=a;` next to `b=[0]; b[0]=b;`, are never the same object, so comparing `a == b` would still recurse forever on identity alone; Python's own list == has exactly this gap, and `a == b` for two such lists raises `RecursionError` in plain Python too. `vertex_equal` additionally tracks which pairs of lists are already being compared, on the path from the outermost call down to the current one; reaching the same pair again means nothing left to compare could ever disagree, so it counts as equal and the recursion stops.

Printing cannot use identity at all, because describing a cyclic list still has to produce something for the element that is the list itself. `vertex_repr` prints `[...]` the second time it reaches a list already on its current path, the same convention Python's own `repr()` uses for a self-referential list. `print(a)` above prints `[[...]]`.

Both mechanisms add a small, constant amount of bookkeeping to every list visited, cyclic or not: one membership test, one set entry added before recursing into that list's elements, and the same entry removed once that list is fully walked. That cost does not grow with how deeply lists are nested, and it does not touch assignment at all, which is where an alternative fix was considered and set aside: rejecting `a[0] = a` outright, at the point of assignment, would mean walking the entire right-hand value of every indexed assignment of a list, looking for a reference back to the list being assigned into, to guard against something that almost never happens. Letting equality and printing tolerate the cycle costs less, not more, and it costs nothing on an assignment that never produces one.

Testing the Contract

`test_lists.py` covers empty and mixed-type literals; element evaluation order; nested literals and chained indexing; every rejected index case (fractional, boolean, string, negative, out of range); indexed assignment, including the out-of-range and non-list-base failures; mutation through an alias versus rebinding leaving the alias untouched; concatenation producing an independent list; structural equality, including the nested `true != 1` regression that Python's own list equality would miss; and `length`, `type`, `print`, and `string` for lists. It also covers self-referential lists: a direct cycle and an indirect, two-list cycle both comparing equal to themselves and printing without crashing, with a non-cyclic case confirming that two separate appearances of the same sublist still print in full rather than collapsing. `exit` is injected at every new position this chapter adds: a list element, an index base, an index, and an indexed assignment's right-hand value, checking both the propagated status and that nothing was mutated along the way.

`test_slice_head_tail.py` covers half-open slices of lists and strings; empty and whole-sequence slices; every rejected bound (fractional, boolean, string, negative, past the end, and start after end); `head` and `tail` on one-element and empty values; new lists from `slice` and `tail` that share their nested elements; `length` of a string; guarded loops that reverse a list and a string; the three new words rejected as names; and `exit` injected at each argument position.

`test_string_indexing.py` covers reading a character, the same rejected indices as for a list (fractional, boolean, string, negative, out of range, and any index into an empty string), indexing a string inside a list, a traversal loop over a string, assignment into a string rejected with the string left unchanged, and `exit` injected into the base and the index.

Practice

1. Write a loop that finds the largest value in a nonempty list using only `while`, `length`, and indexing.
2. Update every element of a list in place by index, then compare the result through a second name that was assigned to the list beforehand. Confirm it sees the same changes.
3. Trace `grid[0][1] = exit(4);` on `grid = [[1, 2], [3, 4]]` by hand before running it. Which parts of the target are evaluated, and in what order, before the status is returned?
4. Using only `head`, `tail`, `+`, and `length`, write a loop that copies a list into a new list one element at a time. Then do the same for a string.
5. Use `slice` to split a list at its midpoint and join the two halves back together with `+`. Try a list with an odd number of elements. What happens, and why?
6. Reverse a string with a loop that reads each character by index. Then try assigning into the string, and explain the error.

Answers to the practice questions are at the end of the chapter.

References

1. Wikipedia contributors. “Array (data type).” Wikipedia. https://en.wikipedia.org/wiki/Array_(data_type)
2. Wikipedia contributors. “Array slicing.” Wikipedia. https://en.wikipedia.org/wiki/Array_slicing

Questions for Thought

1. When should assigning a list copy it, and when should it share the same value? Which did this chapter choose, and why does that choice fall out of decisions made as early as Chapter 2?
2. How can the scalar equality rule `true != 1` affect the equality of two lists that look identical at a glance?
3. What state can change before an indexed assignment encounters `exit`, and what can only change after it?
4. `head` rejects an empty list; `tail` does not. What justifies treating the two differently, and what does `head`’s choice ask of the code that calls it?

Answers to the Practice Questions

1. Start `largest` at the first element, not at a made-up value such as `0`, which would be wrong for a list of negative numbers.

```
values = [3, 1, 4, 1, 5, 9, 2, 6];
largest = values[0];
i = 1;
while (i < length(values)) {
    if (values[i] > largest) {
        largest = values[i];
    };
    i = i + 1;
};
print(largest);
```

   This prints `9`. It needs a nonempty list: `values[0]` on an empty list is an `IndexError`.

2. Each pass adds 5 to one element in place.

```
scores = [70, 80, 90];
alias = scores;
i = 0;
while (i < length(scores)) {
    scores[i] = scores[i] + 5;
    i = i + 1;
};
print(alias);
```

   This prints `[75, 85, 95]`. `alias = scores` made no copy, so there is one list with two names, and the loop's changes show through both.

3. `grid[0][1] = exit(4);` evaluates in this order:

   1. The base `grid[0]`: look up `grid`, evaluate `0`, and read that position, which gives the list `[1, 2]`.
   2. The index `1`.
   3. The check that `1` is a valid position in `[1, 2]`.
   4. The right-hand value `exit(4)`, which returns the status `(4, "exit")`.

   The status propagates and the store never happens, so the program exits with code `4` and `grid` is still `[[1, 2], [3, 4]]`. Step 3 comes before step 4. If the index were out of range, as in `a[5] = exit(4);` on a one-element list, the `IndexError` would be raised and `exit(4)` would never run.

4. Take the first element each pass, append it to the copy, and drop it from what remains.

```
original = [1, 2, 3];
rest = original;
copy = [];
while (length(rest) > 0) {
    copy = copy + [head(rest)];
    rest = tail(rest);
};
print(copy);
```

This prints `[1, 2, 3]`. `copy` is a new list, so `copy[0] = 99;` leaves `original` alone. Only the outer list is new; a list nested inside it would still be shared. The string version is the same loop.

```
word = "hello";
rest = word;
copy = "";
while (length(rest) > 0) {
    copy = copy + head(rest);
    rest = tail(rest);
};
print(copy);
```

This prints `hello`. A string cannot be changed in place, so the copy cannot be told apart from the original.

5. Compute the midpoint, then join the two slices.

```
items = [1, 2, 3, 4];
middle = length(items) / 2;
joined = slice(items, 0, middle) + slice(items, middle, length(items));
print(joined);
```

This prints `[1, 2, 3, 4]`. With four elements, `middle` is `2.0`, a float with no fractional part, which a slice bound accepts. With five elements, `middle` is `2.5` and `slice` raises `TypeError`: slice bounds must be whole numbers. Vertex has no integer division yet, so an odd length has no whole midpoint to compute, and the language rejects `2.5` instead of guessing whether it means `2` or `3`.

6. Each pass puts the next character in front of the result.

```
word = "stressed";
reversed = "";
i = 0;
while (i < length(word)) {
    reversed = word[i] + reversed;
    i = i + 1;
};
print(reversed);
```

This prints `desserts`. Then `word[0] = "S";` raises `TypeError`: only a list can be an assignment target, because a string can be replaced but not changed from the inside. To get the new string, build it: `word = "S" + tail(word);` makes `word` `"Stressed"`.